

Programming with Prolog

Unification and Search

IIIT Bangalore

1 Unification

- Exercises

2 Search

Prolog Terms

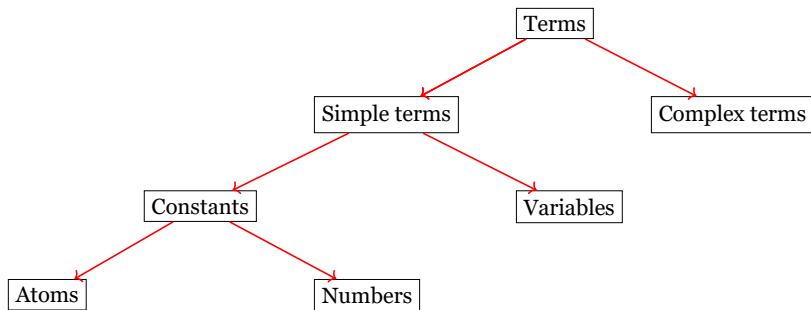
Simple Terms

- Constants
 - Atoms
 - Numbers

- Variables

Complex Terms

- Structured terms with functors



Unification: Definition

Two terms unify if:

- They are identical, or
- Variables can be instantiated to make them identical

Basic Examples

■ `reeta = reeta` $\Rightarrow$ `true` ✓

■ `42 = 42` $\Rightarrow$ `true`

■ `woman(reeta) = woman(reeta)` $\Rightarrow$ `true`

woman
↗
functor

Basic Examples

- `reeta = reeta` $\Rightarrow$ `true`
- `42 = 42` $\Rightarrow$ `true`
- `woman(reeta) = woman(reeta)` $\Rightarrow$ `true`
- `reeta = vijay` $\Rightarrow$ `false`
- `woman(reeta) = woman(jayshree)` $\Rightarrow$ `false`

Unification with Variables

- reeta = X $\Rightarrow$ $X = reeta$
- woman(Z) = woman(reeta) $\Rightarrow$ $Z = reeta$
- loves(reeta,X) = loves(X,vijay)

$X = reeta$
 $X = vijay$ } — X

Revised Definition

- 1 Constants unify if identical ✓
- 2 Variable + term $\Rightarrow$ variable instantiated ✓
- 3 Complex terms unify if:
 - Same functor and arity
 - Arguments unify
 - Substitutions are consistent

Prolog Unification (=/2)

```
?- reeta = reeta.  
true
```

```
?- reeta = vijay.  
false
```

```
?- reeta = X.  
X = reeta
```

Failure Example

```
?- X = reeta, X = vijay.  
false
```

- After first step: $X = reeta$
- Second step fails

Complex Term Example

```
?- k(s(g),Y) = k(X,t(k)).  
X = s(g)  
Y = t(k)
```

Another Example

```
?- loves(X,X) = loves(mohsin,reeta).  
false
```

Occurs Check

- Standard unification uses occurs check
- Prevents infinite terms

```
?- unify_with_occurs_check(father(X), X).  
false
```

Infinite Terms in Prolog

```
?- father(X) = X.  
X = father(father(father(...)))
```

- Prolog omits occurs check by default

Programming with Unification

```
vertical(line(point(X,Y), point(X,Z))).  
horizontal(line(point(X,Y), point(Z,Y))).
```

The diagram illustrates the unification process between two Prolog clauses. The first clause is `vertical(line(point(X,Y), point(X,Z))).` and the second is `horizontal(line(point(X,Y), point(Z,Y))).`. Green arrows show the variable mappings: `X` in the first clause matches `X` in the second; `Y` in the first clause matches `Y` in the second; and `Z` in the first clause matches `Z` in the second. The arrows for `Y` and `Z` are double-lined, indicating a successful match.

Query Examples

```
?- vertical(line(point(1,1),point(1,3))).  
true
```

```
?- vertical(line(point(1,1),point(3,2))).  
false
```

Horizontal Example

- ① Write the predicate to define horizontal line .
- ② Trace the steps followed during the unification that happens ~~with~~ during the evaluation of the following term

```
?- horizontal(line(point(1,1),point(1,Y))).  
Y = 1
```

Proof Search

- Prolog searches knowledge base
- Uses:
 - Unification
 - Backtracking
 - Depth-first search

Example Program

$x=b$
 $x=b$
 $x=b$

$x=a$
 $x=a$
 $x=b$

$x=b$

$f(a)$. $f(b)$.
 $g(a)$. $g(b)$.
 $h(b)$.

$x=y$

$k(X) :- \underline{f(X)}, \underline{g(X)}, \underline{h(X)}.$

Query:

$\exists Y. k(Y) ?$

$?- \underline{k(Y)}.$

Result of Query

```
Y = b ;  
false
```

Another Example

```
loves(vijay,reeta).  
loves(mohsin,reeta).  
  
jealous(A,B):- loves(A,C), loves(B,C).
```

Query

```
?- jealous(X,Y).
```

Query

```
?- jealous(X,Y).
```

```
X = vijay, Y = vijay ;  
X = vijay, Y = mohsin ;  
X = mohsin, Y = vijay ;  
X = mohsin, Y = mohsin ;  
false
```

1 Unification

- Exercises

2 Search

Unification

Which of the following pairs of terms unify? Where relevant, give the variable instantiations that lead to successful unification.

1 `bread = bread`

2 `'Bread' = bread`

3 `'bread' = bread`

4 `Bread = bread`

5 `bread = idli`

6 `food(bread) = bread`

7 `food(bread) = X`

8 `food(X) = food(bread)`

9 `food(bread,X) = food(Y,idli)`

10 `food(bread,X,chai) = food(Y,idli,X)`

11 `food(bread,X,chai) = food(Y,vadapav)`

12 `food(X) = X`

13 `meal(food(bread),drink(chai)) = meal(X,Y)`

14 `meal(food(chai),X) = meal(X,drink(chai))`

1 Unification

- Exercises

2 Search

Search

Example Knowledge Base – Deepak's Family

```
proud(X) :- parent(X, Y), newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

```
? proud(deepak).
```

```
? proud(deepak).
```

```
proud(X) :- parent(X, Y), newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

proud(deepak)

```
? proud(deepak).
```

```
proud(X) :- parent(X, Y), newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

proud(deepak)

|
 $X = \text{deepak}$

```
? proud(deepak).
```

```
proud(X) :- parent(X, Y), newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

```
proud(deepak)  
  |  
  X = deepak  
  |  
parent(deepak, Y)
```

? proud(deepak).

```
proud(X) :- parent(X, Y), newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

proud(deepak)
|
 $X = \text{deepak}$
|
parent(deepak, Y)
|
father(deepak, Y)

```
? proud(deepak).
```

```
proud(X) :- parent(X, Y), newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

```
proud(deepak)  
  |  
  X = deepak  
  |  
parent(deepak, Y)  
  |  
father(deepak, Y)  
  |  
father(deepak, amogh)
```

```
? proud(deepak).
```

```
proud(X) :- parent(X, Y), newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

```
proud(deepak)  
  |  
  X = deepak  
  |  
parent(deepak, Y)  
  |  
father(deepak, Y)  
  |  
father(deepak, amogh)  
  |  
  Y = amogh
```

```
? proud(deepak).
```

```
proud(X) :- parent(X, Y), newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

```
proud(deepak)  
|  
X = deepak  
|  
parent(deepak, Y)  
|  
father(deepak, Y)  
|  
father(deepak, amogh)  
|  
Y = amogh  
|  
✓
```

Search

Search - Deepak's Family

proud(X)

```
proud(X) :- parent(X, Y),  
            newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

Search

Search - Deepak's Family

proud(X)
|
 $X = \text{purnima}$

```
proud(X) :- parent(X, Y),  
            newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

Search

Search - Deepak's Family

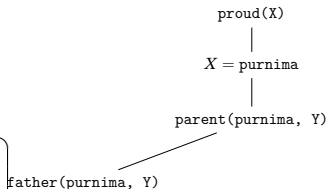
```
proud(X) :- parent(X, Y),  
            newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

```
proud(X)  
  |  
X = purnima  
  |  
parent(purnima, Y)
```

Search

Search - Deepak's Family

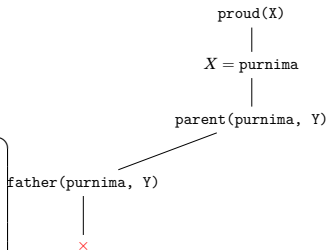
```
proud(X) :- parent(X, Y),  
            newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```



Search

Search - Deepak's Family

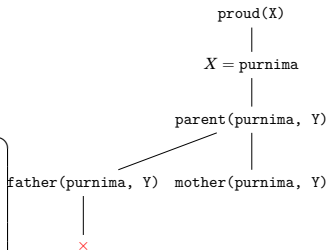
```
proud(X) :- parent(X, Y),  
            newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```



Search

Search - Deepak's Family

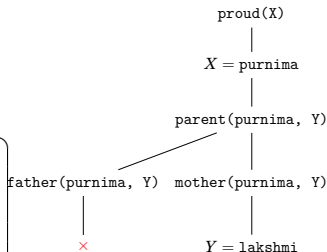
```
proud(X) :- parent(X, Y),  
            newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```



Search

Search - Deepak's Family

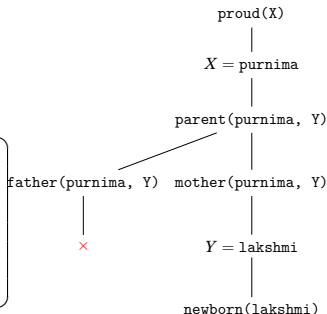
```
proud(X) :- parent(X, Y),  
             newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```



Search

Search - Deepak's Family

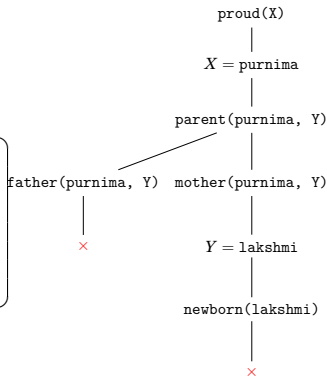
```
proud(X) :- parent(X, Y),
            newborn(Y).
parent(X, Y) :- father(X, Y).
parent(X, Y) :- mother(X, Y).
father(deepak, amogh).
mother(purnima, lakshmi).
mother(purnima, amogh).
newborn(amogh).
```



Search

Search - Deepak's Family

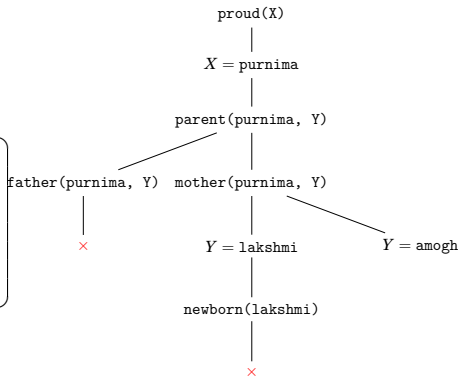
```
proud(X) :- parent(X, Y),  
            newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```



Search

Search - Deepak's Family

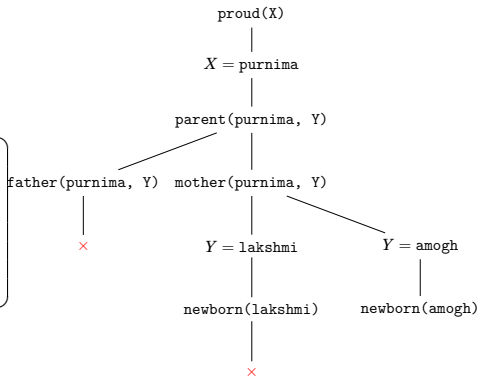
```
proud(X) :- parent(X, Y),  
            newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```



Search

Search - Deepak's Family

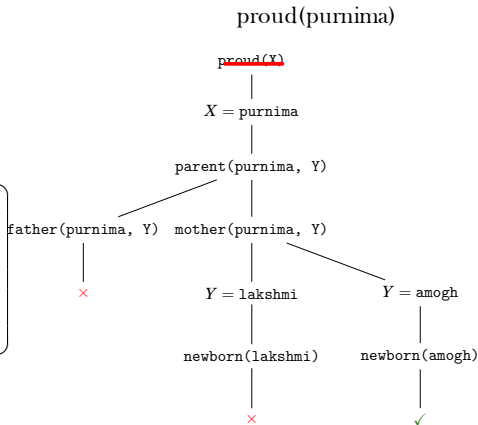
```
proud(X) :- parent(X, Y),  
            newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```



Search

Search - Deepak's Family

```
proud(X) :- parent(X, Y),  
             newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```



Search

Search - Deepak's Family

```
proud(X) :- parent(X, Y),  
            newborn(Y).  
parent(X, Y) :- father(X, Y).  
parent(X, Y) :- mother(X, Y).  
father(deepak, amogh).  
mother(purnima, lakshmi).  
mother(purnima, amogh).  
newborn(amogh).
```

